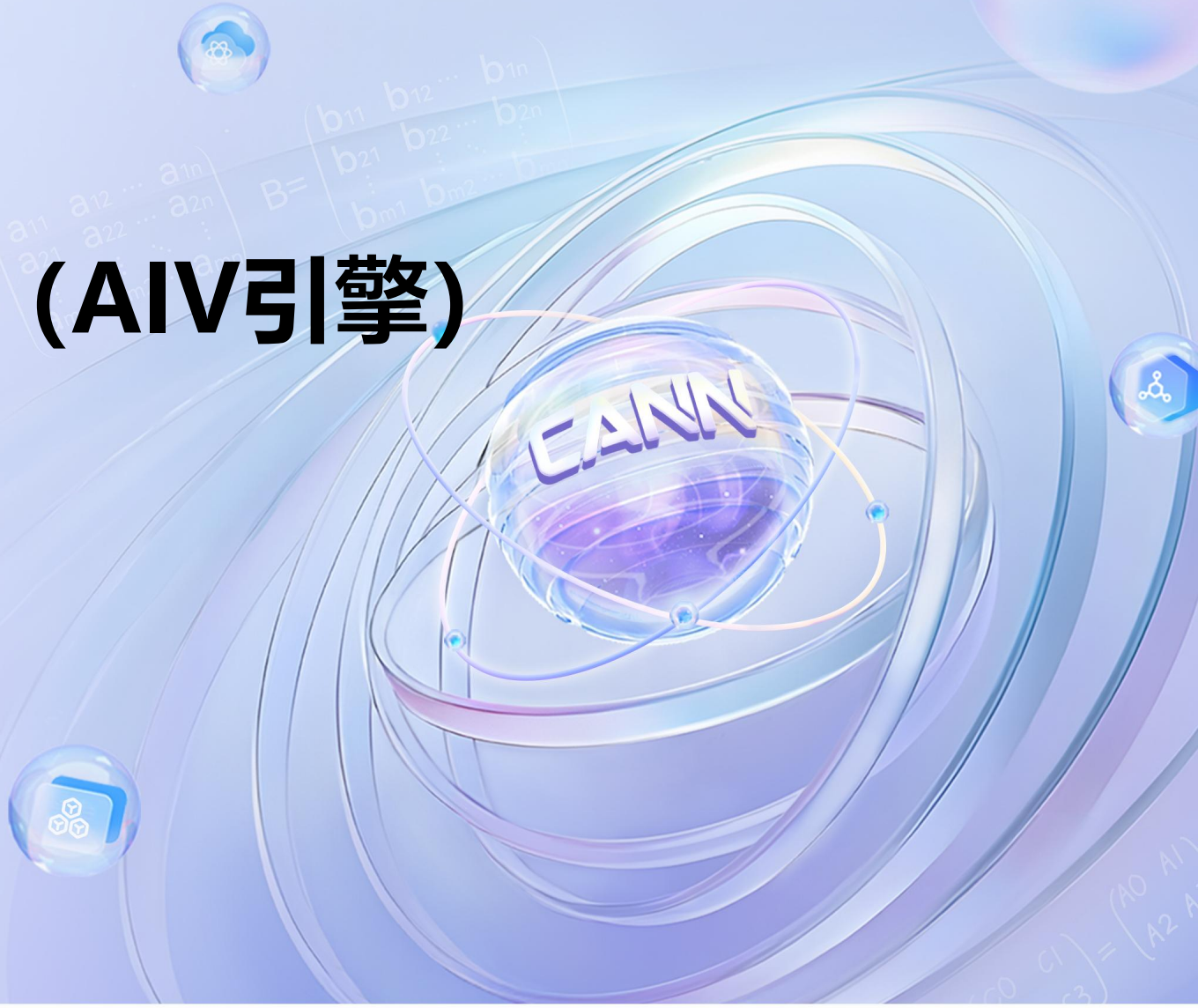
HCCL算子开发入门（AIV引擎）



CANN

修订记录

版本号	修订日期	修订人	修订页码	修订内容摘要
V5.0	2026/9/30	张兴平	全部	初版内容

目录

Part 1 AIV简介

Part 2 AIV编程模型和API介绍（Ascend C）

Part 3 HCCL算子开发-AIV引擎

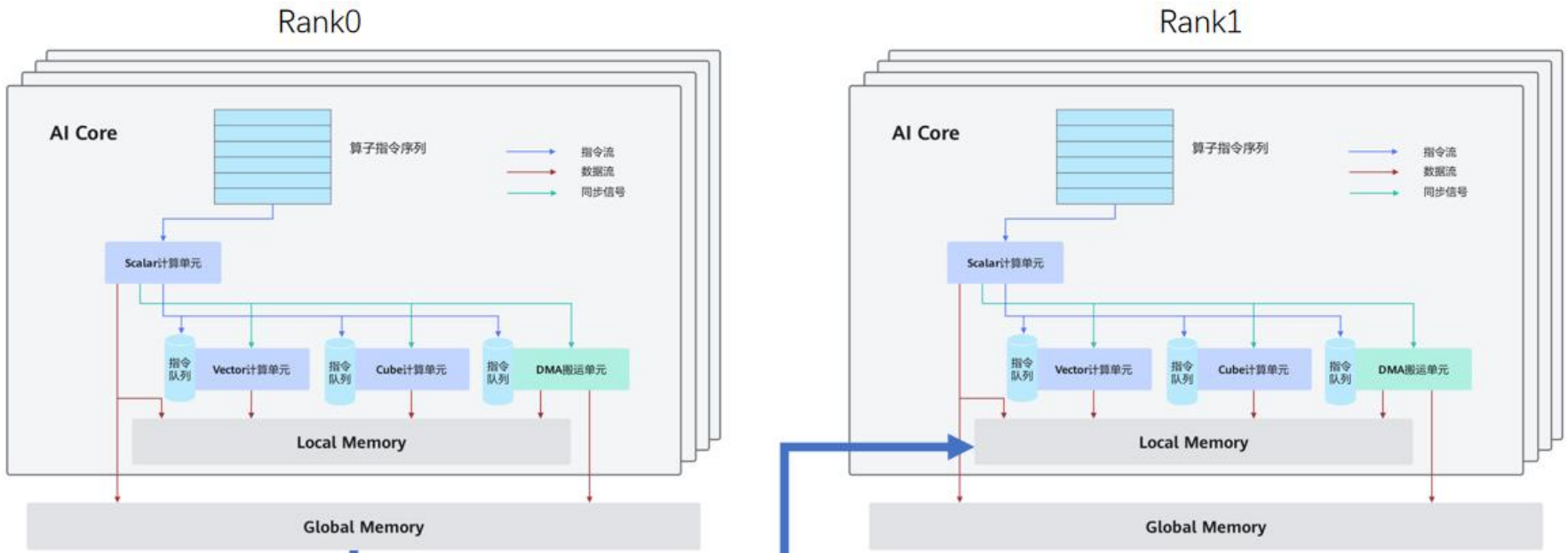
AIV介绍

AI Vector（AIV）是昇腾 AI Core 中的矢量计算单元。AIV 通信引擎利用各 rank 的 Vector Core 执行集合通信任务：本端 AIV 通过片间互联（HCCS/UB 直访）直接读写对端 Device 内存（HCCL Buffer 与软同步标记区），配合软件同步（Notify）协调各 rank 执行时序，完成同步、数据搬运与规约计算，实现集合通信操作。

组件	功能	编程概念
Scalar	控制逻辑：地址计算、循环控制、指令发射	Kernel 控制流、同步 API
Vector（AIV）	矢量运算，承担通信搬运与规约	矢量计算 / 搬运 API
Cube	矩阵运算	矩阵计算 API

解决的问题及优势：

1、标准 Ascend C 矢量编程模型，开发门槛低，直接复用搬运/同步/原子类 API；
2、多 Vector Core 并行执行（SPMD），通信任务切分与编排灵活；
3、数据类型支持丰富，SetAtomic 系列支持 UB→GM 随路 Reduce（SUM/MAX/MIN）；
4、引擎通用性强：Ascend C屏蔽底层芯片差异，A2/A3/950 系列均可用。



AI Core 抽象硬件架构（AIV 即 Vector 单元）

https://git

CANN

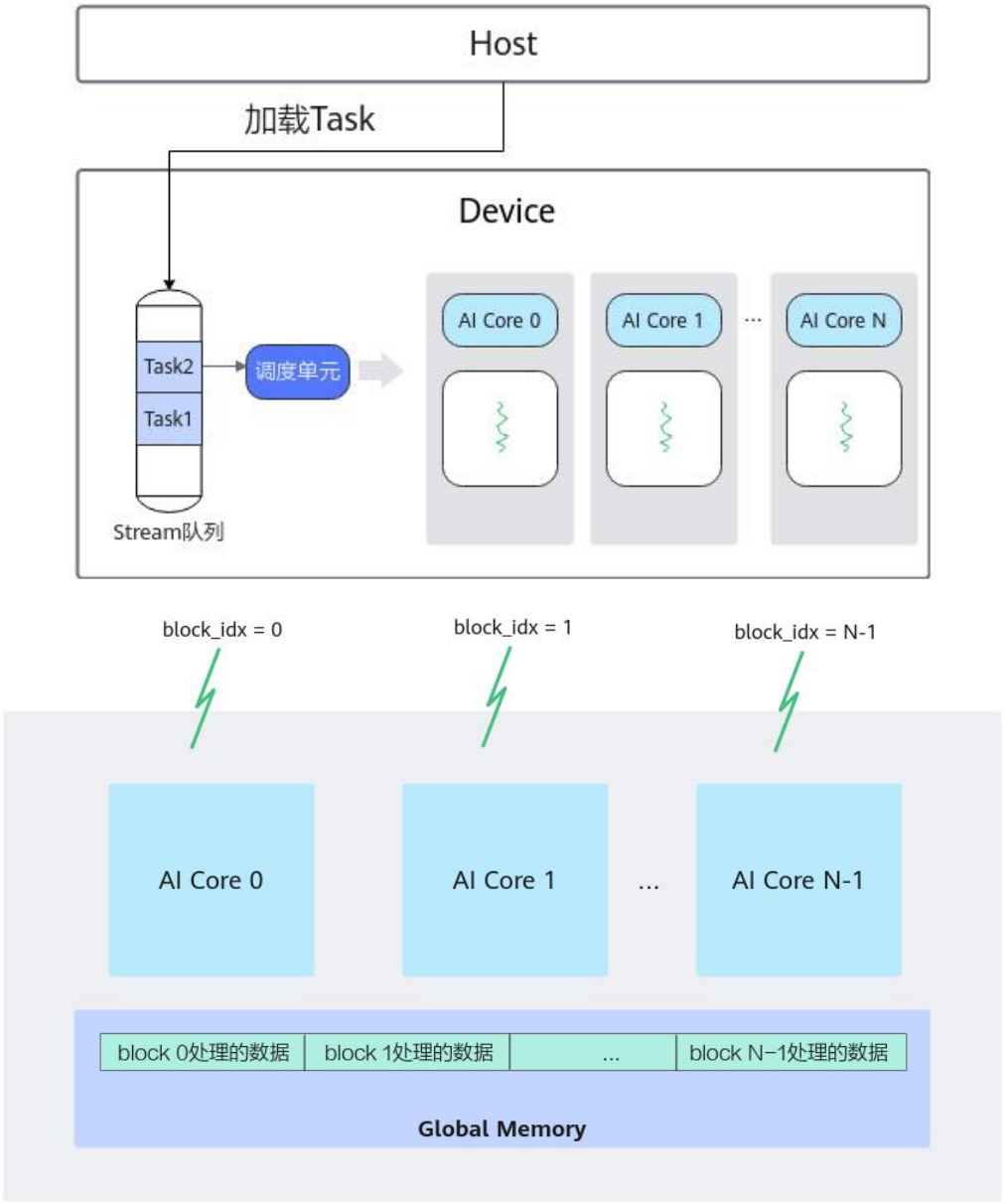
目录

Part 1 AIV简介

Part 2 AIV编程模型和API介绍（Ascend C）

Part 3 HCCL算子开发-AIV引擎

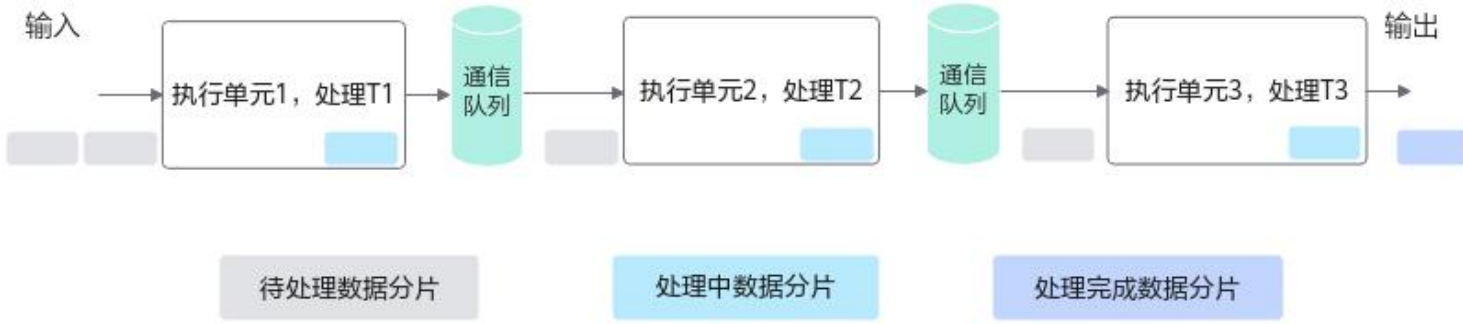
AIV编程模型：硬件模型



SPMD编程模型

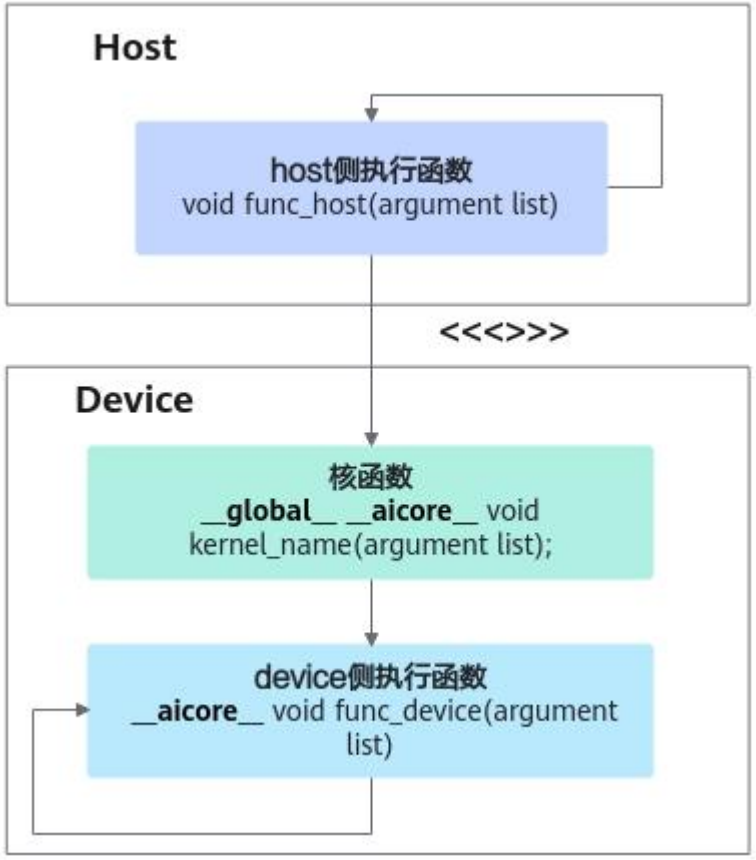
Ascend C算子编程是SPMD（Single-Program Multiple-Data）编程，通俗来讲就是“一份代码，多处执行，处理的数据不同”。SPMD是一种常用的并行计算的方法，是提高计算速度的有效手段。

具体到Ascend C编程模型中的应用，是将需要处理的数据拆分并同时在多个计算核心（类比于上文介绍中的多个进程）上运行，从而获取更高的性能。多个AI Core共享相同的指令代码，每个核上的运行实例唯一的区别是block_idx不同，每个核通过不同的block_idx来识别自己的身份。block的概念类似于上文中进程的概念，block_idx就是标识进程唯一性的进程ID。并行计算过程如下图所示。



CANN

AIV编程模型：核函数



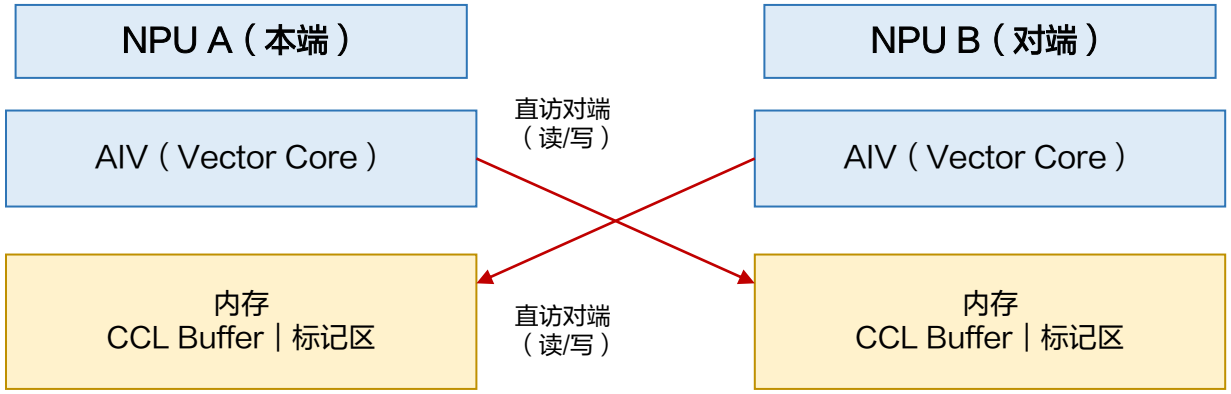
核函数（Kernel Function）是Ascend C算子设备侧实现的入口。Ascend C允许用户使用C/C++函数的语法扩展来编写设备端的运行代码，用户在核函数中进行数据访问和计算操作，由此实现该算子的所有功能。区别于普通的C++函数调用时仅执行一次，当核函数被调用时，多个核都执行相同的核函数代码，具有相同的函数入参，并行执行。

核函数定义时需要使用函数类型限定符__global__和__aicore__；其指针入参变量需要增加变量类型限定符__gm__，表明该指针变量指向Global Memory上某处内存地址；使用<<<...>>>内核调用符调用执行核函数，并指定调用时的执行核数。

核函数（Kernel）：
标准 Ascend C 核函数：extern "C" __global__ __aicore__ void Kernel(param);
SPMD：多核执行同一份代码，GetBlockIdx() 核 ID 区分算法分支；
EXPORT_AIV_META_INFO 声明 AIV 元信息（K_TYPE_AIV）；
编译为独立 .o 二进制，Host 侧运行时加载下发。

CANN

AIV编程模型：数据搬运与计算



AIV 核心能力：跨卡单边读写 + 软同步

使用 DataCopy（对齐搬运）

```
// GM <-> UB（32B 对齐）
DataCopy(dstLocal,
    srcGlobal, count);
DataCopy(dstGlobal,
    srcLocal, count);
```

使用 DataCopyPad（非对齐）

```
// 非对齐补齐
DataCopyPad(dstLocal,
    srcGlobal, cpParam,
    padParam);
```

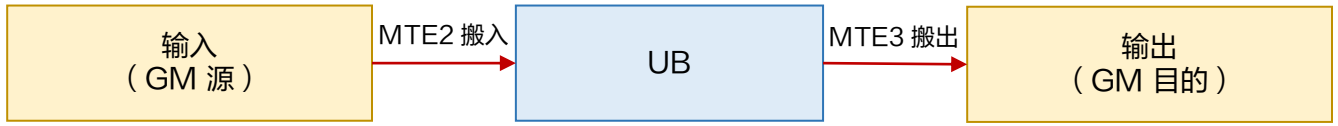
GM2GM（GM→GM 经 UB 中转）

```
// 分块搬运（双缓冲）
LocalTensor<T> in =
    inOutQue.AllocTensor<T>();
DataCopy(in, src, n); // GM2UB
DataCopy(dst, in, n); // UB2GM
inOutQue.FreeTensor(in);
```

数据搬运路径

GM↔UB 经 MTE 搬运单元搬运；GM→GM 需经 UB 中转（GM→UB→GM）；
单块 ≤190KB（双缓冲 95KB×2）；HCCL Buffer 默认 200MB，超限分块处理。

随路 Reduce（UB→GM 传输时开启）：
SUM → SetAtomicAdd；MAX → SetAtomicMax；MIN → SetAtomicMin；完成后 SetAtomicNone。



补充说明：

- 双缓冲：TQueBind<VECIN, VECOUT> 搬入 / 搬出流水重叠；
- count 超过单块上限时自动分块循环搬运。

AIV编程模型：同步与通信

软同步（Notify）

```
// Record: 向 rank 标记区写入 tag
__aicore__ inline void Record(
    uint32_t rank, uint64_t off, int32_t tag) {
    localTagTensor.SetValue(0, tag);  // UB 置标记
    DataCopy(d2hGlobal, localTagTensor, ...); // 写对端标记区UB2GM
}
// Wait: 轮询标记区直到等于 tag
__aicore__ inline void Wait(
    uint32_t rank, uint64_t off, int32_t tag) {
    while (true) {
        DataCopy(localTagTensor, d2hGlobal, ...); // 读标记区GM2UB
        if (localTagTensor.GetValue(0) == tag) // 标量比较
            break;
    }
}
```

- Record：标记值 UB→GM，写对端标记区；
- Wait：轮询 GM→UB 拷回并标量比较；
- tag 随通信轮次递增，防新旧标志混淆（回绕 4096）；
- 核内同步：PipeBarrier；核间同步：SyncAll / SyncAllSafe

补充说明：

- 软同步（Record/Wait）与数据搬运底层均为 DataCopy

通信内存与地址交换（Host 侧）



- HcclGetHcclBuffer / HcclChannelGetHcclBuffer：获取本端 / 远端 CCL Buffer；
- HcclChannelGetRemoteMems：获取远端标记区地址；地址表 buffersInOut 经 aclrtMemcpy 写入 Device。

```
// Kernel 侧: InitBuffArray 读地址表
GM_IN[i]  = aivCommInfoMem[2*i];     // rank i 的 CCL Buffer
GM_OUT[i] = aivCommInfoMem[2*i+1];   // rank i 的标记区
```

地址表（AivCommInfo）模型

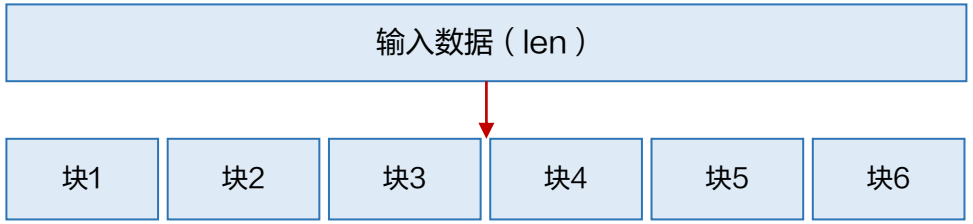
CANN

AIV编程模型：多核并行与分块

AIV 并发模型

多核切分数据、分块流水搬运，提升带宽利用率

```
// InitCoreInfo：按 coreId 均分数据（余数给前若干核）
uint64_t dataPerCore = len / coreCount;
uint64_t remainder   = len % coreCount;
if (coreId < remainder) {
    curCount   = dataPerCore + 1;
    coreOffset = coreId * (dataPerCore + 1) * sizeof(T);
} else {
    curCount   = dataPerCore;
    coreOffset = remainder * (dataPerCore + 1) * sizeof(T)
               + (coreId - remainder) * dataPerCore * sizeof(T);
}
```



每块 ≤ cclBufferSize（32B 对齐）

逐块 curTag+1，BatchRecordWait 后进下一块

大数据分块（Streaming）

多核切分（SPMD）：

- 多核执行同一 Kernel，GetBlockIdx() 核 ID 区分数据片；
- numBlocks ≥ rankSize：每核独立处理一片（Run）；
- 核数不足：RunCtrlCore 两阶段（各核写入 → 汇聚 → 分摊拷出）；
- 双缓冲：TQueBind<VECIN, VECOUT> 搬入 / 搬出重叠。

大数据分块：

- Process 按 cclBufferSize 对齐切块，逐块处理；
- 块间 BatchRecordWait 串行；输入 / 输出指针随块步进。

CANN

AIV编程接口

```
// Kernel 定义与使用示例
extern "C" __global__ __aicore__
void HcclAllGatherAivKernel(OpParam param) {
    switch (param.dataType) {
        // int8 / int32 / fp16 / fp32
        case AscendC::HCCL_DATA_TYPE_FP32:
            CALL_AIV_KERNEL(float);
            break;
        ...
    }
}
EXPORT_AIV_META_INFO(
    HcclAllGatherAivKernel);
```

AIV数据面接口（Ascend C，Kernel 侧）

接口	说明
DataCopy / DataCopyPad	GM↔UB 数据搬运（对齐 / 非对齐补齐）
SetFlag / WaitFlag	同步原语（用于AIV内的流水间同步）
PipeBarrier / SyncAll	核内流水同步 / 核间同步
SetAtomicAdd / Max / Min	UB→GM 随路 Reduce（SUM/MAX/MIN）
GetBlockIdx / block_num	核 ID / 核数（SPMD 分支依据）
TPipe / TBuf / TQue	UB 缓冲管理与双缓冲流水

AIV控制面接口（Host 侧）

接口	说明
HcclEngineCtxGet / Create	获取 / 创建 Context（标记区）
HcclCommMemReg	内存注册到通信域
HcclRankGraphGetLinks	查询拓扑链路
HcclChannelAcquire	建立通信 Channel
HcclGetHcclBuffer	获取本端 CCL Buffer
HcclChannelGetRemoteMems	获取远端内存地址
aclrtBinaryLoad / GetFunction	加载 Kernel 二进制
aclrtLaunchKernelWithHostArgs	下发 Kernel（AIV 引擎）

CANN

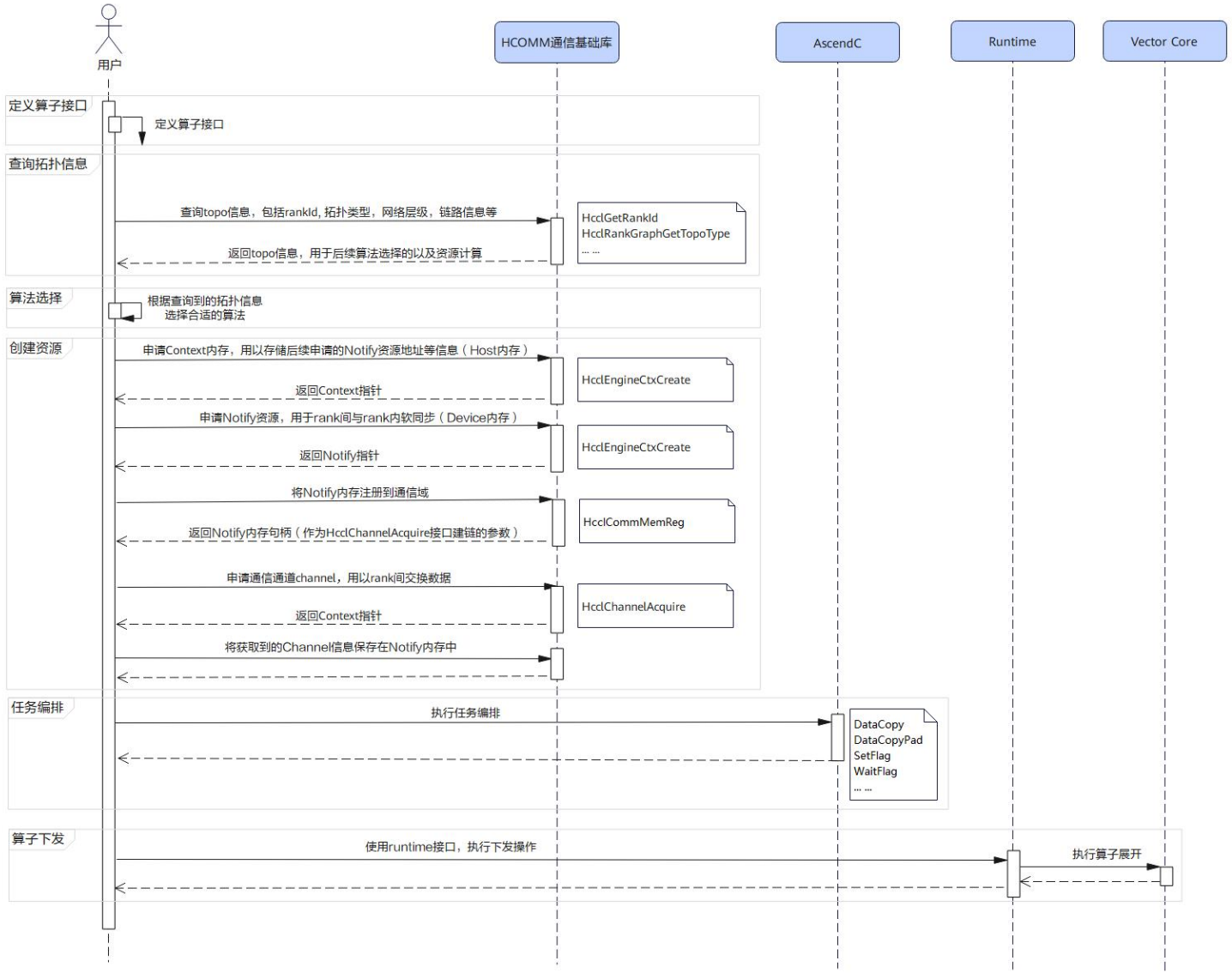
目录

Part 1 AIV简介

Part 2 AIV编程模型和API介绍（Ascend C）

Part 3 HCCL算子开发-AIV引擎

HCCL算子执行流程：AIV通信引擎



算子控制面（算子入口）：

- 执行算法选择
- 申请&管理资源（标记区 / Channel / 地址表）
- 注册 Kernel 二进制（全局一次）
- 下发 AIV Kernel 到引擎执行

算子数据面（AIV Kernel）：

- 基于 Ascend C 实现数据搬运与软同步
- 多 Vector Core SPMD 执行，按核 ID 分工
- 前同步 → 跨卡读写 → 后同步
- Kernel 编译为 .o，Host 加载、AIV 引擎执行

CANN

HCCL算子开发：AIV AllGather算子样例

算子控制面开发

```
// allgather.cc：资源准备（PrepareResources）
// 1. 创建软同步标记区（Notify）并注册到通信域
HcclEngineCtxCreate(comm, tag, COMM_ENGINE_AIV,
                    AIV_TAG_BUFF_LEN, &aivCommInfoPtr);
aclrtMemset(aivCommInfoPtr, AIV_TAG_BUFF_LEN, 0,
            AIV_TAG_BUFF_LEN);
HcclCommMemReg(comm, tag, &regMem, &memHandle);
// 2. 查询拓扑，按 P2P 协议逐对端建立 Channel
HcclRankGraphGetLinks(comm, layer, rank, remoteRank,
                      &linkList, &listSize);   // UB/HCCS/SIO
desc.notifyNum = 3;
HcclChannelAcquire(comm, COMM_ENGINE_AIV,
                   descs, validNum, channels);
// 3. 收集本端 / 远端通信内存地址
HcclGetHcclBuffer(comm, &cclBufferAddr, &cclBufferSize);
HcclChannelGetHcclBuffer(comm, ch, &remoteAddr, &size);
HcclChannelGetRemoteMems(comm, ch, &memNum, &mems, &tags);
buffersInOut[2*r]     = cclBufferAddr;  // 各 rank CCL 槽
buffersInOut[2*r + 1] = tagBufAddr;     // 各 rank 标记区
// 4. 地址表同步至 Device，供 Kernel 读取
aclrtMemcpy(aivCommInfoPtr, bytes, buffersInOut,
            bytes, ACL_MEMCPY_HOST_TO_DEVICE);
```

要点：

- 标记区 / 地址表共用同一块 Context 内存（AivCommInfo）；
- Channel 建立时完成地址交换，Kernel 只读表；
- 二进制全局加载一次，后续算子调用直接下发。

```
// launch_kernel.cc：Kernel 注册与下发
// 1. 加载 Kernel 二进制（全局一次）
LoadBinaryFromFile("hccl_custom_allgather_kernels.o",
                   binBuffer, binSize);
aclrtBinaryLoad(...) -> g_binHandle;
aclrtBinaryGetFunction(g_binHandle,
    "HcclAllGatherAivKernel", &g_funcHandle);
// 2. 下发 Kernel 到 AIV 引擎
aclrtLaunchKernelAttr attr[3];
attr[0] = SCHEM_MODE;
attr[1] = TIMEOUT_US;   // 超时保护
attr[2] = ENGINE_TYPE = ACL_RT_ENGINE_TYPE_AIV;
aclrtLaunchKernelWithHostArgs(g_funcHandle, 1, stream,
    &cfg, &param, sizeof(OpParam), nullptr, 0);
```

下发属性（attr）：

- SCHEM_MODE：调度模式；
- TIMEOUT_US：超时保护，避免软同步死等（通过HCCL_EXEC_TIMEOUT环境变量可配置超时时间）；
- ENGINE_TYPE = ACL_RT_ENGINE_TYPE_AIV：指定 AIV 引擎执行。

https://gitcode.com/cann

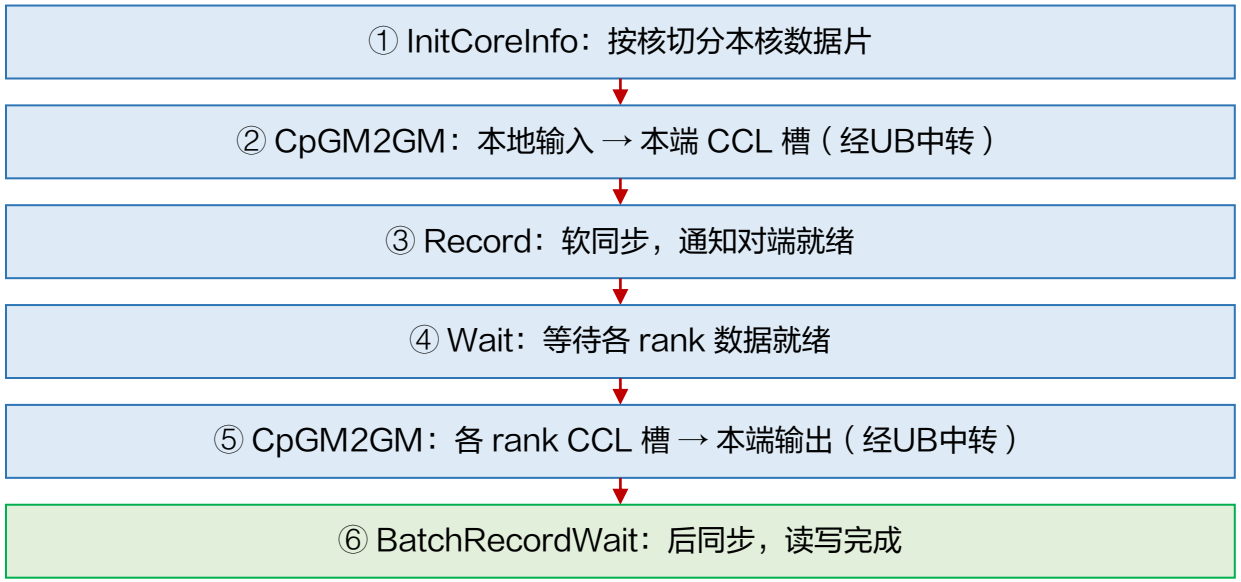
CANN

参考链接：https://gitcode.com/cann/hccl/blob/master/examples/05_custom_ops_allgather/aiv/op_host/allgather.cc

HCCL算子开发：AIV AllGather算子样例

算子数据面（AIV Kernel）开发

```
// launch_kernel_asc.asc：Kernel 入口
extern "C" __global__ __aicore__
void HcclAllGatherAivKernel(OpParam param) {
    switch (param.dataType) {   // int8/int32/fp16/fp32
        ... CALL_AIV_KERNEL(float); ...
    }
}
// aiv_allgather_mesh_1d.h：Mesh 1D 算法
InitCoreInfo(count, curTag);                  // ① 按核切分
CpGM2GM(GM_IN[rank_]+off, input_+off, n); // ② 输入→本端CCL槽
Record(rank_, blockIdx*FLAG_SIZE, curTag);// ③ 通知对端就绪
for (r = 0; r < rankSize_; ++r) {
    Wait(r, blockIdx*FLAG_SIZE, curTag);  // ④ 等待就绪
    CpGM2GM(output_+r*stride+off,              // ⑤ 拷至输出
            GM_IN[r]+off, n);
}
BatchRecordWait(curTag);                      // ⑥ 后同步
// 大数据量：Process 按 cclBufferSize 分块，逐块 curTag+1
// 首算子防串扰：BarrierForFirstOP；收尾：BarrierAll
```



（多核并行：每核独立执行 ①–⑥）

执行要点：

- 前同步：BarrierForFirstOP，首个算子跨卡栅栏防串扰；
- tag 管理：GetTag 从标记区读取并递增，防新旧混淆；
- 收尾：BarrierAll 核间栅栏；
- Reduce 类算子：CpGM2GM(…, atomicOp) 开启随路规约。

https://gitcode.com/cann

参考链接：https://gitcode.com/cann/hccl/blob/master/examples/05_custom_ops_allgather/aiv/op_kernel/launch_kernel_asc.asc

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN